dYdX v4 Protocol Layer - CLOB 모듈 상세 구현 분석

개요

CLOB (Central Limit Order Book) 모듈은 dYdX v4 Protocol의 핵심 구성 요소로, 중앙 지정가 주문서를 관리하고 고성능 매칭 엔진을 구현합니다. 이 문서는 CLOB 모듈의 아키텍처, 매칭 엔진 알고리즘, MemCLOB 인메모리 주문서 관리 메커니즘, 그리고 블록 생명주기에서의 역할을 상세히 분석합니다.

CLOB 모듈 아키텍처

전체 구조

```
protocol/x/clob/
├── keeper/ # 상태 관리 및 비즈니스 로직
├── memclob/ # 인메모리 주문서 구현
├── types/ # 데이터 타입 및 인터페이스
├── ante/ # 트랜잭션 전처리
├── rate_limit/ # 속도 제한
├── flags/ # 설정 플래그
├── abci.go # ABCI 생명주기 메서드
├── module.go # 모듈 정의
```

```
└── genesis.go # Genesis 상태 관리
```

핵심 컴포넌트

1. Keeper (상태 관리자)

위치: `protocol/x/clob/keeper/keeper.go`

역할: CLOB 모듈의 핵심 비즈니스 로직과 상태 관리

주요 필드:

```
type Keeper struct {
cdc codec.BinaryCodec
storeKey storetypes.StoreKey
memKey storetypes.StoreKey
transientStoreKey storetypes.StoreKey
// 인메모리 주문서
MemClob types.MemClob
// 조건부 주문 관리
UntriggeredConditionalOrders map[types.ClobPairId]*UntriggeredConditionalOrders
// 영구 선물 ID와 CLOB 쌍 매핑
PerpetualIdToClobPairId map[uint32][]types.ClobPairId
// 다른 모듈 Keeper들
subaccountsKeeper types.SubaccountsKeeper
assetsKeeper types.AssetsKeeper
perpetualsKeeper types.PerpetualsKeeper
```

```
pricesKeeper  types.PricesKeeper

// ... 기타 Keeper들

// 속도 제한

placeOrderRateLimiter  rate_limit.RateLimiter[*types.MsgPlaceOrder]

cancelOrderRateLimiter  rate_limit.RateLimiter[*types.MsgCancelOrder]

// 청산 정보

DaemonLiquidationInfo *liquidationtypes.DaemonLiquidationInfo

}
```

2. MemCLOB (인메모리 주문서)

위치: `protocol/x/clob/memclob/memclob.go`

역할: 고성능 인메모리 주문서 관리

주요 구조:

```
type  MemClobPriceTimePriority  struct {

// 열린 주문 관리

openOrders *memclobOpenOrders

// 취소된 주문 관리

cancels *memclobCancels

// 다음 블록에 제안할 작업들

operationsToPropose  types.OperationsToPropose

// CLOB Keeper 참조

clobKeeper  types.MemClobKeeper

// 오프체인 업데이트 생성 여부
```

```
generateOffchainUpdates  bool

}
```

매칭 엔진 알고리즘 (가격-시간 우선순위)

매칭 우선순위 규칙

1. **가격 우선순위**: 더 좋은 가격의 주문이 먼저 매칭

- 매수 주문: 높은 가격 우선
- 매도 주문: 낮은 가격 우선

2. **시간 우선순위**: 같은 가격에서는 먼저 들어온 주문이 우선

3. **주문 타입별 처리**:

- **Short-Term Orders**: 메모리에서 즉시 처리
- **Long-Term Orders**: 상태에 저장 후 처리
- **Conditional Orders**: 조건 만족 시 트리거

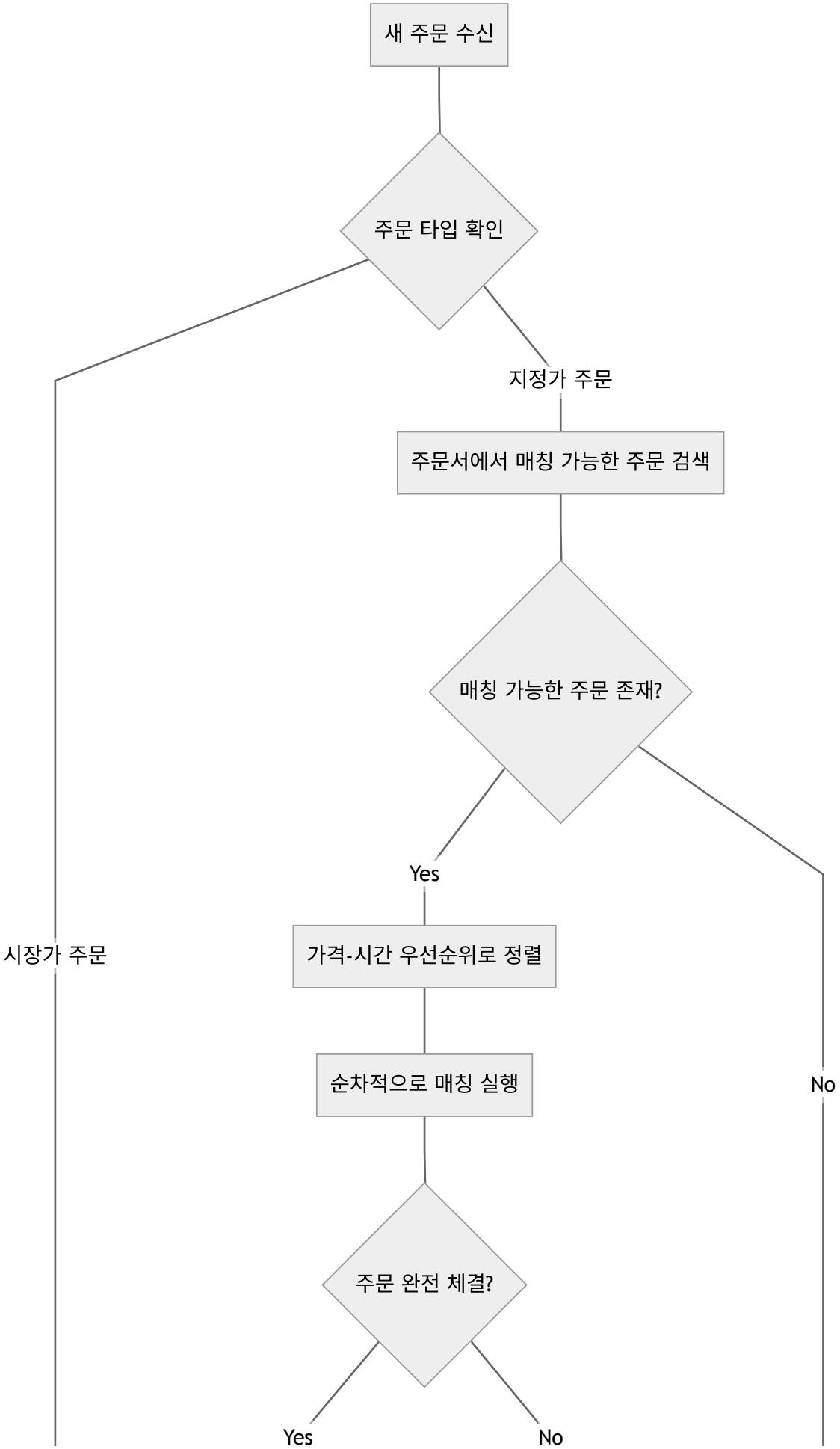
매칭 프로세스

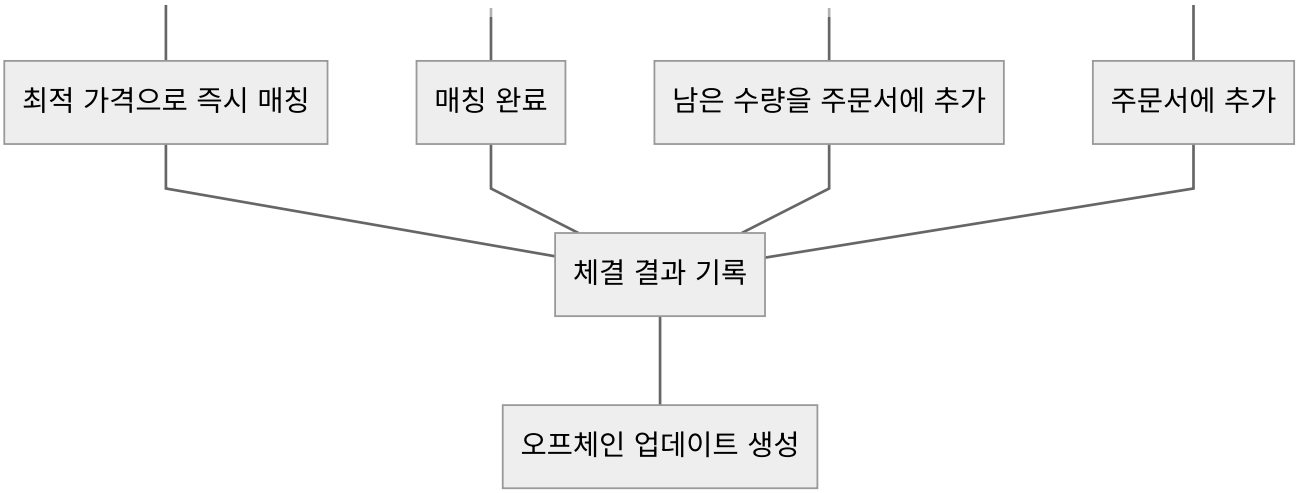
1. 주문 배치 (Order Placement)

```
// PlaceOrder 함수의 핵심 로직

func (m *MemClobPriceTimePriority) PlaceOrder(

ctx  sdk.Context,

order  types.Order,

) (
```

```
orderSizeOptimisticallyFilledFromMatchingQuantums satypes.BaseQuantums,
orderStatus types.OrderStatus,
offchainUpdates *types.OffchainUpdates,
err error,
) {
// 1. 주문 검증
if err := m.validatePlaceOrder(ctx, order); err != nil {
return 0, types.Undercollateralized, nil, err
}
// 2. 매칭 시도
matchedOrders, remainingOrder := m.attemptMatching(ctx, order)
// 3. 매칭 결과 처리
for _, match := range matchedOrders {
m.processMatch(ctx, match)
}
// 4. 남은 주문을 주문서에 추가
if remainingOrder != nil {
m.addToOrderbook(ctx, remainingOrder)
}
return orderSizeOptimisticallyFilledFromMatchingQuantums, orderStatus, offchain
}
```

2. 매칭 알고리즘


새 주문 수신
주문 타입 확인
지정가 주문
주문서에서 매칭 가능한 주문 검색
매칭 가능한 주문 존재?
Yes
가격-시간 우선순위로 정렬
순차적으로 매칭 실행
주문 완전 체결?
Yes
No
No
시장가 주문



3. 매칭 검증

매칭이 발생하기 전에 다음 검증을 수행합니다:

1. **계정 검증**: 충분한 잔고 및 마진 확인
2. **주문 유효성**: 만료 시간, 주문 크기 등 확인
3. **시장 상태**: CLOB 쌍 상태 및 거래 가능 여부 확인
4. **리스크 관리**: 포지션 한도 및 리스크 매개변수 확인

MemCLOB 인메모리 주문서 관리 메커니즘

데이터 구조

1. 주문서 구조

```
type memclobOpenOrders struct {
// CLOB 쌍별 주문서 맵
orderbooksMap map[types.ClobPairId]*Orderbook
// 주문 ID로 주문 검색을 위한 맵
```

```
orderIdToLevelOrder  map[types.OrderId]*list.Element
}

type  Orderbook  struct {
// 매수/매도 주문 레벨
Bids *OrderbookSide
Asks *OrderbookSide
// 서브계정별 열린 주문
SubaccountOpenClobOrders  map[satypes.SubaccountId]map[types.OrderSide]map[types
// CLOB 쌍 설정
ClobPair  types.ClobPair
}
```

2. 가격 레벨 관리

```
type  OrderbookSide  struct {
// 가격별 주문 레벨 (정렬된 맵)
Levels  map[types.Subticks]*Level
// 가격 정렬을 위한 키 슬라이스
LevelKeysAsc []types.Subticks  // 오름차순 (매도용)
LevelKeysDesc []types.Subticks  // 내림차순 (매수용)
}
```

```
type Level struct {
// 해당 가격의 주문들 (시간 순서)
Orders *list.List
// 총 수량
TotalQuantums satypes.BaseQuantums
}
```

주문서 작업

1. 주문 추가

```
func (m *MemClobPriceTimePriority) addOrderToOrderbook(
ctx sdk.Context,
order types.Order,
) {
clobPairId := order.GetClobPairId()
orderbook := m.openOrders.orderbooksMap[clobPairId]
// 가격 레벨 가져오기 또는 생성
level := orderbook.getOrCreateLevel(order.GetOrderSide(), order.GetSubticks())
// 주문을 레벨에 추가 (시간 순서 유지)
element := level.Orders.PushBack(order)
// 빠른 검색을 위한 맵 업데이트
m.openOrders.orderIdToLevelOrder[order.GetOrderId()] = element
// 서브계정 주문 추적 업데이트
```

```
m.updateSubaccountOrders(order, true)

}
```

2. 주문 제거

```
func (m *MemClobPriceTimePriority) mustRemoveOrder(

ctx  sdk.Context,

orderId  types.OrderId,

) {

// 주문 찾기

element, exists := m.openOrders.orderIdToLevelOrder[orderId]

if !exists {

return

}

order := element.Value.(types.Order)

// 레벨에서 주문 제거

level := m.getLevel(order.GetClobPairId(), order.GetOrderSide(), order.GetSubtic

level.Orders.Remove(element)

// 레벨이 비어있으면 제거

if  level.Orders.Len() == 0 {

m.removeLevel(order.GetClobPairId(), order.GetOrderSide(), order.GetSubticks())

}

// 맵에서 제거

delete(m.openOrders.orderIdToLevelOrder, orderId)
```

```
// 서브계정 주문 추적 업데이트
m.updateSubaccountOrders(order, false)
}
```

메모리 최적화

1. **객체 풀링**: 자주 생성/삭제되는 객체의 재사용
2. **지연 삭제**: 즉시 삭제 대신 마킹 후 배치 삭제
3. **압축된 데이터 구조**: 메모리 사용량 최소화
4. **캐시 지역성**: 관련 데이터를 인접하게 배치

블록 생명주기에서의 CLOB 역할

BeginBlocker

위치: `protocol/x/clob/abci.go`

실행 시점: 각 블록의 시작

주요 작업:

```
func BeginBlocker(ctx sdk.Context, keeper types.ClobKeeper) {
// 1. 이전 블록의 매칭 이벤트 초기화
keeper.MustSetProcessProposerMatchesEvents(
ctx,
types.ProcessProposerMatchesEvents{
BlockHeight: lib.MustConvertIntegerToUint32(ctx.BlockHeight()),
```

```
},

)

}
```

EndBlocker

실행 시점: 각 블록의 끝

주요 작업:

```
func EndBlocker(ctx sdk.Context, keeper keeper.Keeper) {

// 1. 만료된 Short-Term 주문의 체결 정보 정리

keeper.PruneStateFillAmountsForShortTermOrders(ctx)

// 2. 만료된 Stateful 주문 제거

expiredStatefulOrderIds := keeper.RemoveExpiredStatefulOrdersTimeSlices(ctx, ctx

// 3. 만료된 조건부 주문 정리

keeper.PruneUntriggeredConditionalOrders(expiredStatefulOrderIds, ...)

// 4. 새로 배치된 조건부 주문 추가

keeper.AddUntriggeredConditionalOrders(ctx, ...)

// 5. 조건부 주문 트리거 확인

triggeredConditionalOrderIds := keeper.MaybeTriggerConditionalOrders(ctx)

// 6. 속도 제한 정보 정리

keeper.PruneRateLimits(ctx)

// 7. 메트릭 업데이트

metrics.SetGauge(
```

```
metrics.InsuranceFundBalance,

metrics.GetMetricValueFromBigInt(keeper.GetCrossInsuranceFundBalance(ctx)),

)

}
```

PrepareCheckState

실행 시점: 다음 블록 준비 시

주요 작업:

```
func PrepareCheckState(ctx sdk.Context, keeper *keeper.Keeper) {

// 1. 로컬 검증자의 작업 큐에서 작업 제거

localValidatorOperationsQueue, shortTermOrderTxBytes := keeper.MemClob.GetOperat

keeper.MemClob.RemoveAndClearOperationsQueue(ctx, localValidatorOperationsQueue)

// 2. MemCLOB에서 무효한 상태 정리

offchainUpdates := keeper.MemClob.PurgeInvalidMemclobState(ctx, ...)

// 3. 이전 블록에서 배치된 Stateful 주문들을 MemCLOB에 배치

offchainUpdates = keeper.PlaceStatefulOrdersFromLastBlock(ctx, ..., offchainUpda

// 4. 트리거된 조건부 주문들을 MemCLOB에 배치

offchainUpdates = keeper.PlaceConditionalOrdersTriggeredInLastBlock(ctx, ..., of

// 5. 로컬 검증자의 작업들을 다시 실행

replayUpdates := keeper.MemClob.ReplayOperations(ctx, ...)

// 6. 청산 가능한 서브계정 청산 시도

liquidatableSubaccountIds := keeper.DaemonLiquidationInfo.GetLiquidatableSubacco
```

```
subaccountsToDeleverage, err := keeper.LiquidateSubaccountsAgainstOrderbook(ctx,

// 7. 서브계정 디레버리징

if err := keeper.DeleverageSubaccounts(ctx, subaccountsToDeleverage); err != nil

panic(err)

}

// 8. 부정적 TNC 서브계정에 대한 출금 제한

negativeTncSubaccountIds := keeper.DaemonLiquidationInfo.GetNegativeTncSubaccoun

if err := keeper.GateWithdrawalsIfNegativeTncSubaccountSeen(ctx, negativeTncSuba

panic(err)

}

// 9. 오프체인 메시지 전송

keeper.SendOffchainMessages(offchainUpdates, nil, metrics.SendPrepareCheckStateO

// 10. 새로운 gRPC 스트림 초기화

keeper.InitializeNewGrpcStreams(ctx)

// 11. 주문서별 메트릭 설정

keeper.MemClob.SetMemclobGauges(ctx)

}
```

주문 타입별 처리

1. Short-Term Orders (단기 주문)

특징:

- 메모리에만 저장
- 블록 번호로 만료 시간 지정
- 빠른 처리 속도

처리 과정:

```
func (k Keeper) PlaceShortTermOrder(ctx sdk.Context, msg *types.MsgPlaceOrder) e

order := msg.GetOrder()

// 1. 기본 검증

if err := k.validateShortTermOrder(ctx, order); err != nil {

return err

}

// 2. MemCLOB에 직접 배치

_, orderStatus, offchainUpdates, err := k.MemClob.PlaceOrder(ctx, order)

// 3. 오프체인 업데이트 전송

k.SendOffchainMessages(offchainUpdates, nil, metrics.SendPlaceOrderOffchainUpdat

return err

}
```

2. Long-Term Orders (장기 주문)

특징:

- 상태에 영구 저장
- 시간 기반 만료
- 블록 간 지속성

처리 과정:

```
func (k Keeper) PlaceLongTermOrder(ctx sdk.Context, msg *types.MsgPlaceOrder) er

order := msg.GetOrder()

// 1. 상태에 주문 저장

k.SetLongTermOrderPlacement(ctx, order, ctx.BlockHeight())

// 2. 다음 블록의 PrepareCheckState에서 MemCLOB에 배치될 예정

// (즉시 MemCLOB에 배치하지 않음)

return nil

}
```

3. Conditional Orders (조건부 주문)

특징:

- 특정 조건 만족 시 활성화
- 트리거 가격 기반
- 복잡한 거래 전략 지원

처리 과정:

```
func (k Keeper) MaybeTriggerConditionalOrders(ctx sdk.Context) []types.OrderId {

var triggeredOrderIds []types.OrderId

// 모든 CLOB 쌍에 대해 조건부 주문 확인

for clobPairId, untriggeredOrders := range k.UntriggeredConditionalOrders {

// 현재 시장 가격 가져오기
```

```
marketPrice := k.getMarketPrice(ctx, clobPairId)

// 트리거 조건 확인

for orderId, order := range untriggeredOrders.Orders {

if k.shouldTriggerOrder(order, marketPrice) {

// 주문을 상태에 저장

k.SetLongTermOrderPlacement(ctx, order, ctx.BlockHeight())

triggeredOrderIds = append(triggeredOrderIds, orderId)

// 트리거되지 않은 주문 목록에서 제거

delete(untriggeredOrders.Orders, orderId)

}

}

}

return triggeredOrderIds

}
```

리스크 관리 및 청산 통합

청산 프로세스

CLOB 모듈은 청산 데몬과 긴밀하게 연동하여 리스크 관리를 수행합니다:

```
func (k Keeper) LiquidateSubaccountsAgainstOrderbook(

ctx sdk.Context,

liquidatableSubaccountIds []satypes.SubaccountId,
```

```
) ([]types.SubaccountLiquidationInfo, error) {

var subaccountsToDeleverage []types.SubaccountLiquidationInfo

for _, subaccountId := range liquidatableSubaccountIds {

// 1. 서브계정의 포지션 및 마진 확인

positions := k.subaccountsKeeper.GetAllPositions(ctx, subaccountId)

// 2. 청산 가능한 포지션 식별

for _, position := range positions {

if k.shouldLiquidatePosition(ctx, subaccountId, position) {

// 3. 청산 주문 생성 및 실행

liquidationOrder := k.createLiquidationOrder(ctx, subaccountId, position)

// 4. MemCLOB에서 매칭 시도

_, _, _, err := k.MemClob.PlaceOrder(ctx, liquidationOrder)

if err != nil {

// 매칭 실패 시 디레버리징 대상에 추가

subaccountsToDeleverage = append(subaccountsToDeleverage,

types.SubaccountLiquidationInfo{

SubaccountId: subaccountId,

Position: position,

})

}

}

}

}
```

```
    return subaccountsToDeleverage, nil

}
```

성능 최적화

1. 메모리 관리

- **객체 재사용**: 자주 생성되는 객체의 풀링
- **가비지 컬렉션 최소화**: 불필요한 할당 방지
- **메모리 지역성**: 관련 데이터의 인접 배치

2. 알고리즘 최적화

- **이진 검색**: 가격 레벨 검색 최적화
- **지연 평가**: 필요할 때만 계산 수행
- **배치 처리**: 여러 작업을 한 번에 처리

3. 동시성 처리

- **읽기 최적화**: 읽기 작업의 병렬 처리
- **락 최소화**: 세밀한 락 전략
- **원자적 연산**: 경쟁 조건 방지

결론

dYdX v4의 CLOB 모듈은 고성능 탈중앙화 거래소의 핵심 요구사항을 충족하기 위해 정교하게 설계되었습니다. 가격-시간 우선순위 기반의 매칭 엔진, 효율적인 인메모리 주문서 관리, 그리고 블록체인 생명주기와의 긴밀한 통합을 통해 기존 중앙화 거래소에 필적하는 성능과 기능을 제공합니다. 특히

MemCLOB의 인메모리 처리와 다양한 주문 타입 지원, 그리고 리스크 관리 시스템과의 통합은 dYdX v4가 고빈도 거래와 복잡한 거래 전략을 지원할 수 있게 하는 핵심 기술입니다.